

Implementation Plan: Ninja Analyzer — 7 Phases

Context

The analyzer at `apps/analyzer/` has powerful SEO engines (scan, audit, GSC, opportunities, strategies, work plans, execution) but the operator must sit in a browser clicking through tabs. The product vision: **operator adds keywords → system works → operator approves from phone → system executes → operator sees results.**

7 phases to get there:

#	Phase	Effort	What it unlocks
0	Foundation (job queue + notifications)	1-2d	Durable background work, no lost jobs
1	Telegram Bot (read + alerts)	2-3d	Operator sees everything from phone
2	Keyword Discovery + Auto-Pipeline	2-3d	System finds keywords + auto-analyzes on add
3	Telegram Approvals	1-2d	Operator approves from phone (no execute yet)
4	AI Content Writer	2-3d	Actual content from briefs via Claude
5	Content Calendar	2-3d	Scheduled, paced publishing
6	PageSpeed Dashboard	1-2d	Speed scores + history (PSI calls already exist)

Total: ~12-18 days sequential, ~10-14 with Phase 6 in parallel

Phase 0: Foundation (1-2 days)

Why first: Every subsequent phase needs durable background jobs and a clean notification layer. Without this, Telegram webhook fire-and-forget will lose jobs on Vercel timeout.

Migration: `add_foundation`

```
model PipelineRun {
  id          String    @id @default(cuid())
  clientId    String?
  type        String    // keyword_refresh | full_refresh | scan | gsc_sync | speed_audit |
content_generate
  status      String    @default("queued") // queued | running | success | failed
  triggeredBy String    // telegram | ui | cron | keyword_add
  payload     String?   // JSON input params
  result      String?   // JSON output summary
  error       String?
  createdAt   DateTime  @default(now())
  startedAt   DateTime?
  finishedAt  DateTime?
  @@index([status, createdAt])
  @@index([clientId, type])
  @@schema("analyzer")
}

model BotNotification {
  id          String    @id @default(cuid())
  chatId      String
```

```

    messageId   String? // Telegram message ID (for editing)
    type        String   // scan_result | opps_found | plan_ready | keyword_discovery |
execution_result | impact_review | daily_digest | calendar_reminder
    clientId    String?
    referenceId String? // ID of the related entity (planId, oppId, etc.)
    status      String   @default("sent") // sent | edited | expired
    sentAt      DateTime @default(now())
    @@index([chatId, type])
    @@schema("analyzer")
}

```

New files

File	Purpose
src/lib/jobs.ts	Types: PipelineRunType, JobPayload, JobResult
src/lib/jobs-server.ts	enqueueJob(type, clientId, payload, triggeredBy) → creates PipelineRun, processJob(jobId) → dispatches to the right function, drainJobs() → picks queued jobs with per-client lock
src/app/api/jobs/drain/route.ts	Worker route: maxDuration = 300, auth via CRON_SECRET, picks + processes queued jobs. Called by cron or self-invoked from webhook
src/lib/notify.ts	notifyOperator(type, data) — unified notification dispatch. Calls Telegram (Phase 1), logs to BotNotification, replaces ad-hoc notification calls

Modify existing files

File	Change
vercel.json	Add /api/jobs/drain cron (every 1 min or on-demand)

Key design

- **Per-client lock:** PipelineRun with status = "running" acts as a lock. drainJobs() skips clients with a running job of the same type.
- **Idempotent:** If a job already exists for (clientId, type, status=queued), don't create a duplicate.
- **Self-invoking:** Webhook can fetch("/api/jobs/drain") to wake the worker immediately after enqueueing, without waiting for cron.

Phase 1: Telegram Bot — Read + Alerts (2-3 days)

Read-only commands + proactive notifications. No mutations yet.

New files

File	Purpose
src/lib/telegram.ts	Bot API client — sendMessage(), editMessage(), answerCallbackQuery(), inline keyboard builder. Raw fetch(), no npm dep
src/lib/telegram-format.ts	Format engine results as Telegram HTML. One formatter per engine output. All respect 4096-char limit with graceful truncation
src/lib/telegram-commands.ts	Parse /command <args> → route to read-only queries

src/app/api/telegram/webhook/route.ts	POST webhook. Validates X-Telegram-Bot-API-Secret-Token header + checks TELEGRAM_ALLOWED_USER_IDS. Returns 200 immediately. Enqueues jobs for long ops via Phase 0
---------------------------------------	--

Commands (read-only)

Command	What it does
/clients	List all clients with health scores
/status <client>	Client overview: findings, opps, GSC freshness, health
/opps <client>	Top opportunities (read-only, no buttons yet)
/plan <client>	Active work plan summary
/help	Command list

Proactive notifications (via notifyOperator())

Hook into existing pipelines:

Trigger point	Notification
refreshClient() completes	Summary: GSC rows, opps found, strategies computed
runAgencyAutoSync() completes	Agency digest: clients processed, findings
logExecutionEvent() fires	Execution alerts (reuse existing severity/dedupe logic)

Modify existing files

File	Change
src/lib/refresh-server.ts	Call notifyOperator("refresh_complete", result) at end
src/lib/automation-server.ts	Call notifyOperator("agency_sync_complete", result) at end
src/lib/execution-events-server.ts	Add Telegram channel to dispatch alongside Slack/email

Env vars

TELEGRAM_BOT_TOKEN , TELEGRAM_CHAT_ID , TELEGRAM_ALLOWED_USER_IDS , TELEGRAM_WEBHOOK_SECRET , TELEGRAM_ENABLED

Security

- Validate X-Telegram-Bot-API-Secret-Token header (set via setWebhook())
- Check message.from.id against TELEGRAM_ALLOWED_USER_IDS
- Log all commands to BotNotification for audit trail

Phase 2: Keyword Discovery + Auto-Pipeline (2-3 days)

Migration: add_keyword_discovery

```
model KeywordSuggestion {
  id                String    @id @default(cuid())
  clientId          String
```

```

query                String
normalizedQuery       String
page                 String?
clicks28d            Int      @default(0)
impressions28d       Int      @default(0)
ctr                  Float?
position             Float?
trend                String? // up | down | flat | unknown
score                Int      @default(0) // 0-100 priority
intent               String?
reason               String   // Hebrew: why this is worth tracking
status               String   @default("suggested") // suggested | approved | rejected |
converted
  convertedKeywordId String?
  createdAt           DateTime @default(now())
  updatedAt           DateTime @updatedAt
  @@unique([clientId, normalizedQuery])
  @@index([clientId, status, score(sort: Desc)])
  @@schema("analyzer")
}

```

New files

File	Purpose
src/lib/keyword-discovery.ts	Types, scoring constants, filter rules
src/lib/keyword-discovery-server.ts	discoverKeywords(clientId) — compare GSC queries vs keyword bank, score + filter, upsert KeywordSuggestion rows

discoverKeywords() rules

1. Load all GSC queries (last 28 days, aggregated)
2. Load keyword bank (normalized)
3. For each GSC query NOT in bank:
 - Skip if <100 impressions (no real demand)
 - Skip if brand/navigational query (heuristic: contains client domain/name)
 - Skip if ranks on non-SEO-eligible page (scope check)
 - Skip if position <4 (already winning, low priority)
 - Skip if position >50 (too far, low confidence)
4. Score remaining: `impressions_weight + position_proximity + ctr_gap + trend_bonus`
5. Infer intent (reuse `inferIntent()` from briefs-server)
6. Upsert to `KeywordSuggestion`
7. Notify operator: "X new keyword suggestions for [client]" + top 5 with Add/Ignore buttons

Auto-Pipeline on Keyword Add

Modify existing files

File	Change
src/app/clients/[id]/keywords/actions.ts	After <code>addKeyword()</code> / <code>addKeywordsBulk()</code> , enqueue a <code>keyword_refresh</code> job via Phase 0. For bulk: use <code>createManyAndReturn()</code> (Prisma 7 supports it) or post-insert query to get IDs
src/lib/refresh-server.ts	New <code>refreshKeywords(clientId, keywordIds[])</code> : sync GSC if stale → <code>computeKeywordStrategy()</code> per keyword → <code>resolveAllMasterPages()</code> →

generate briefs if needed → buildSeoWorkPlan()

Hook into refresh pipeline

Add `discoverKeywords(clientId)` as a step in `refreshClient()` (after opportunity analysis, before strategy compute).

Telegram integration

- Keyword discovery notification with inline buttons: `[+ Add]` `[▶▶ Skip]` per suggestion
- Auto-pipeline completion notification with strategy summary

Phase 3: Telegram Approvals (1-2 days)

Now add mutation buttons — but NO execute. Approve/reject only.

Add to `telegram-commands.ts`

Command	What it does
<code>/scan <client></code>	Enqueue scan job → notify when done
<code>/sync</code>	Enqueue GSC sync → notify when done
<code>/refresh <client></code>	Enqueue full refresh → notify when done

Inline keyboard callbacks

Callback	Action	Safety
<code>approve_opp:<id></code>	Set opportunity status to approved	Validate opp exists + is in approvable state
<code>reject_opp:<id></code>	Set to rejected	Same validation
<code>approve_group: <planId>:<group></code>	Call <code>approveWorkPlanGroup()</code>	Validate plan is active + group is approvable. Prepare-only — no dry run, no execute, no plugin writes
<code>add_keyword: <suggestionId></code>	Convert <code>KeywordSuggestion</code> → <code>TargetKeyword</code> → trigger auto-pipeline	Validate suggestion exists + status=suggested
<code>ignore_keyword: <suggestionId></code>	Set suggestion to rejected	Same

What's NOT in Telegram (intentionally)

- **No execute button** — execution requires seeing the full diff. Link to dashboard: "Dry run ready → [View in Dashboard](#)"
- **No rollback button** — too dangerous for a chat interface
- **No bulk execute** — one at a time, from the web UI

Stale callback protection

Before acting on any callback, validate:

- The referenced entity still exists
- Its status allows the action (e.g., plan not superseded)
- If stale, `answerCallbackQuery("This item was already updated")` and edit the message to show current state

Phase 4: AI Content Writer (2-3 days)

Migration: `add_content_drafts`

New model (not extra columns on ContentBrief — keeps concerns separate):

```
model ContentDraft {
  id          String   @id @default(cuid())
  briefId     String
  brief       ContentBrief @relation(fields: [briefId], references: [id], onDelete: Cascade)
  version     Int       @default(1)
  model       String    // claude-sonnet-4-6, etc.
  promptHash  String?   // hash of system+user prompt for cache/dedup
  content     String    // generated text
  wordCount   Int
  inputTokens Int?
  outputTokens Int?
  status      String    @default("draft") // draft | review | approved | rejected
  feedback    String?   // operator feedback for regeneration
  approvedAt  DateTime?
  approvedBy  String?
  createdAt   DateTime @default(now())
  @@index([briefId, version(sort: Desc)])
  @@schema("analyzer")
}
```

New files

File	Purpose
src/lib/ai-writer.ts	Types, word count targets by brief type, prompt templates
src/lib/ai-writer-server.ts	generateContent(briefId, feedback?) → build prompt → Claude API → store ContentDraft

Modify existing files

File	Change
src/app/clients/[id]/briefs/actions.ts	generateContentAction(), regenerateContentAction(), approveContentAction()
src/app/clients/[id]/briefs/page.tsx	"Generate" button, content preview drawer, version history, word count
src/lib/telegram-commands.ts	/write <briefId> → enqueue content generation job → notify with preview
package.json	Add @anthropic-ai/sdk

Claude API details

- Model: configurable via `ANTHROPIC_MODEL` env var, default `claude-sonnet-4-6`
- `max_tokens` : 8192 (not 4096 — Hebrew articles can run long, and we want room for revision metadata)
- System prompt: vertical, brand voice, language (Hebrew), target audience, SEO guidelines
- User prompt: keyword, intent, outline from brief, secondary keywords, internal links, CTA, competitor context
- Content always `status: "draft"` — no auto-publish
- For YMYL verticals (medical/legal/finance): auto-flag as `status: "review"` with warning
- Revision history: each regeneration creates a new ContentDraft with incremented version

Env vars

ANTHROPIC_API_KEY , ANTHROPIC_MODEL (optional, defaults to claude-sonnet-4-6)

Phase 5: Content Calendar (2-3 days)

Migration: add_content_calendar

```
model ContentSchedule {
  id          String  @id @default(cuid())
  clientId    String
  client      Client  @relation(fields: [clientId], references: [id], onDelete: Cascade)
  briefId     String  @unique
  brief       ContentBrief @relation(fields: [briefId], references: [id], onDelete: Cascade)
  scheduledDate DateTime
  status      String  @default("scheduled") // scheduled | published | skipped | cancelled
  publishedAt DateTime?
  publishedBy String?
  notes       String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  @@index([clientId, scheduledDate])
  @@schema("analyzer")
}
```

Add to Client : publishingCadence Int @default(2) , publishingDays String[]

New files

File	Purpose
src/lib/calendar.ts	Types, Hebrew day/month labels
src/lib/calendar-server.ts	scheduleContent(), getCalendar(), autoSchedule(), getUpcomingWeek(), publishScheduledContent()
src/app/clients/[id]/calendar/page.tsx	Month grid, color-coded by status
src/app/clients/[id]/calendar/actions.ts	Server actions
src/app/api/cron/content-calendar/route.ts	Daily cron — notify today's scheduled content

Modify existing files

File	Change
src/app/clients/[id]/layout.tsx	Add "לוח תוכן" tab
src/app/clients/[id]/settings/ProfileForm.tsx	Cadence + publishing days
src/lib/work-plan-server.ts	After group approval, auto-schedule new briefs if cadence > 0
src/lib/telegram-commands.ts	/calendar <client> command
vercel.json	Add daily calendar cron

Auto-scheduling algorithm

- 1. Load cadence (e.g., 2/week) + preferred days (e.g., Sunday, Wednesday)
- 2. Generate 4-week slot grid, skip filled slots
- 3. Load eligible briefs (approved content draft exists, no schedule yet) ordered by priority
- 4. Assign one per slot, priority-first
- 5. Notify operator: "Scheduled X posts for [client] over next 4 weeks"

Phase 6: PageSpeed Dashboard (1-2 days)

Can run in parallel with Phases 4-5. PSI API calls already exist in `tech-audit-server.ts:583` — this phase persists history + adds a dashboard.

Migration: `add_pagespeed_history`

```
model PageSpeedScore {
  id                String    @id @default(cuid())
  clientId          String
  client            Client    @relation(fields: [clientId], references: [id], onDelete: Cascade)
  pageUrl           String
  strategy           String    // mobile | desktop
  performanceScore  Float     // 0-100
  lcp               Float?    // Largest Contentful Paint (ms)
  inp               Float?    // Interaction to Next Paint (ms) — replaces FID
  cls               Float?    // Cumulative Layout Shift
  fcp               Float?    // First Contentful Paint (ms)
  ttfb              Float?    // Time to First Byte (ms)
  speedIndex        Float?
  recommendations   String?   // JSON
  fetchedAt         DateTime  @default(now())
  @@unique([clientId, pageUrl, strategy, fetchedAt])
  @@index([clientId, fetchedAt(sort: Desc)])
  @@schema("analyzer")
}
```

Note: uses `inp` (Interaction to Next Paint), not `fid` — Google replaced FID with INP in March 2024.

New files

File	Purpose
<code>src/lib/pagespeed.ts</code>	Types, CWV thresholds
<code>src/lib/pagespeed-server.ts</code>	<code>persistSpeedScores()</code> (store results from existing PSI calls), <code>getSpeedSummary()</code> , <code>getSpeedHistory()</code> , <code>runFullSpeedAudit()</code> (expand to 20 pages)
<code>src/app/clients/[id]/speed/page.tsx</code>	Dashboard: CWV breakdown, page table, history charts
<code>src/app/clients/[id]/speed/actions.ts</code>	Server actions

Modify existing files

File	Change
<code>src/lib/tech-audit-server.ts</code>	After existing PSI calls, persist results to <code>PageSpeedScore</code>

src/app/clients/[id]/layout.tsx	Add "מהירות" tab
src/lib/refresh-server.ts	Expand PSI to 20 pages during full refresh if PSI_ENABLED
src/lib/telegram-commands.ts	/speed <client> command

Rate limiting

PSI API: 400/day with key. Cap 20 pages/client. Mobile-only for cron, desktop on manual "Run Audit."

Timeline

Phase 0 (Foundation)	==[1-2d]==>	
Phase 1 (Telegram Read)	===[2-3d]===>	
Phase 2 (Discovery+Pipeline)	===[2-3d]===>	
Phase 3 (Telegram Approvals)	==[1-2d]==>	
Phase 4 (AI Writer)	===[2-3d]===>	
Phase 5 (Calendar)	===[2-3d]===>	
Phase 6 (PageSpeed)	==[1-2d]==>	(parallel w/ 4-5)
	---- ---- ---- ---- ---- ---- ----	
	d1 d3 d5 d7 d9 d11 d13	

Migrations Summary (all additive, non-breaking)

- 1. add_foundation — PipelineRun + BotNotification
- 2. add_keyword_discovery — KeywordSuggestion
- 3. add_content_drafts — ContentDraft (separate from ContentBrief)
- 4. add_content_calendar — ContentSchedule + Client publishing fields
- 5. add_pagespeed_history — PageSpeedScore

Key Design Decisions (informed by Codex review)

- 1. **Durable job queue over fire-and-forget** — PipelineRun table with per-client locking. Webhook enqueues, worker route drains. No untracked promises.
- 2. **Telegram approvals are prepare-only** — approve/reject opportunities and work plan groups. No execute or rollback from chat. Execution requires viewing the full diff in the dashboard.
- 3. **Separate ContentDraft from ContentBrief** — version history, model tracking, token/cost logging, feedback loop. Brief stays clean as the "spec," draft is the "output."
- 4. **KeywordSuggestion as its own model** — not jammed into Opportunity. Has its own lifecycle: suggested → approved → converted to TargetKeyword → triggers auto-pipeline.
- 5. **INP not FID** — Google replaced First Input Delay with Interaction to Next Paint in March 2024.
- 6. **PSI integration extends existing code** — tech-audit-server.ts already calls PSI. Phase 6 persists results + adds dashboard, not building from zero.
- 7. **AI model is configurable** — ANTHROPIC_MODEL env var. YMYL content auto-flagged for review.
- 8. **Stale callback protection** — every Telegram button callback validates current DB state before acting.

Verification

Phase 0

- Enqueue a job, verify it appears in PipelineRun with status=queued
- Hit `/api/jobs/drain`, verify job runs and status=success
- Enqueue duplicate job for same client+type, verify dedup
- Verify per-client lock prevents concurrent runs

Phase 1

- Create bot via @BotFather, set webhook with secret token
- Send each command, verify formatted response
- Trigger a refresh, verify proactive notification arrives
- Send from unauthorized user, verify rejection

Phase 2

- Run `discoverKeywords()` on a client with GSC data, verify suggestions scored correctly
- Add keyword via UI, verify `keyword_refresh` job enqueued and strategy computed
- Bulk add 20 keywords, verify ONE job (not 20)
- Verify Telegram notification with discovery summary

Phase 3

- Tap approve button in Telegram, verify opportunity status changes
- Tap approve on a stale/superseded plan, verify graceful error
- Verify no execute buttons exist in any Telegram message

Phase 4

- Generate content for a brief, verify Hebrew quality
- Regenerate with feedback, verify new version created
- Verify YMYL brief auto-flagged as "review"

Phase 5

- Auto-schedule 5 briefs, verify even spread on preferred days
- Verify daily cron sends Telegram reminder

Phase 6

- Run speed audit, verify scores persisted
- Run again after a week, verify historical chart shows both data points